

XML:

XML stands for eXtensible Markup Language and is a text-based markup language derived from Standard Generalized Markup Language (SGML). It is a markup language which is software- and hardware-independent tool for storing and transporting data. You can write a program in any language on any platform (Operating System) to send, receive or store data using XML.

It is called extensible because it allows the author of the document to define the markup elements by their own. It is a markup language to develop documents containing structured information. By saying that, it is meant that xml contains content (i.e. text, images etc.) and along with that it also contains some information or hints about what role that content plays. **XML is recommended by W3C.**

History of XML

XML started way back in 1996 and was first published in 1998. World Wide Web Consortium (W3C) is the developer of XML, and it became a W3C recommendation in 1998.

There are two versions of XML.

1. XML 1.0
2. XML 1.1

XML 1.1 is the latest version. Yet, XML 1.0 is the most used version.

Simple Example of XML

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student>
    <name>Rick Grimes</name>
    <age>35</age>
    <subject>Maths</subject>
    <gender>Male</gender>
  </student>
  <student>
    <name>Daryl Dixon </name>
    <age>33</age>
    <subject>Science</subject>
    <gender>Male</gender>
  </student>
  <student>
    <name>Maggie</name>
    <age>36</age>
    <subject>Arts</subject>
    <gender>Female</gender>
  </student>
</students>
```

Features/Properties of XML

- It is extensible and human-readable. It allows you to create your own self-descriptive tags, or language, that suits your application
- It was developed by an organization called the World Wide Web Consortium (W3C) and is available as an open standard.
- Well-structured format is easy to read and write from programs.
- It focuses on data rather than how the data looks and it separates data from HTML.
- It is designed to send, store, receive and display data. In simple words you can say that XML is used for storing and transporting data.
- It is platform and language independent.
- It preserves white space.
- Self-descriptive nature.
- Unlike HTML where most of the tags are predefined, XML doesn't have predefined tags, rather you have to create your own tags.

Applications of XML

Web Services

One of the most common use cases for XML is in web services, such as SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) APIs. XML serves as a standard data format for exchanging information between different systems, allowing seamless communication and interoperability.

Document Storage and Management

XML is widely used for storing and managing documents, such as Microsoft Office files (e.g., Word, Excel, and PowerPoint). XML provides a standardized way to represent the document structure, formatting, and metadata, enabling cross-platform compatibility and easy processing by software applications.

Configuration Files

XML is a popular choice for storing application configuration data. It offers a human-readable format that is easy to edit and manage, while also being easily parsed and processed by software applications. XML-based configuration files can be found in many applications, such as Apache web servers and Android app development.

Data Serialization and Deserialization / Data Exchange

XML is often used as a data interchange format for serialization and deserialization of data between applications. It allows complex data structures, like objects and arrays, to be represented in a standardized and easily readable format.

RSS Feeds

RSS (Really Simple Syndication) is an XML-based format for sharing and distributing frequently updated content, like news articles, blog posts, and podcasts. RSS feeds allow

users to stay up-to-date with their favorite websites without having to visit each site individually.

GIS and Geospatial Data

XML is commonly used in the Geographic Information System (GIS) domain for encoding and exchanging geospatial data. Notable XML-based standards include the Geography Markup Language (GML) and Keyhole Markup Language (KML).

Enhancing Data Availability

XML increases data availability by allowing different applications to access your data, not only in HTML pages but also from XML data sources. This makes your data available to all kinds of "reading machines" (Handheld computers, voice machines, news feeds, etc.), and makes it more accessible for blind people or people with other disabilities

Differences Between HTML and XML

1. XML is a text-based markup language which has the self-describing structure and can effectively define another markup language. On the other hand, HTML is a predefined markup language and has a limited capability.
2. XML provides logical structuring of the document while HTML structure is predefined where "head" and "body" tags are used.
3. When it comes to language type HTML is case insensitive. As against, XML is case sensitive.
4. HTML was designed with the emphasis on the presentational features of the data. In contrast, XML is data specific where the data storage and transfer was the prior concern.
5. XML does not permit any mistake if there are some errors in the code it could not be parsed. Inversely, in HTML small errors can be neglected.
6. Whitespaces in XML are used for a specific use as XML considers every single character. On the contrary, HTML can ignore the whitespaces.
7. The tags in XML are mandatory to be closed, whereas in HTML an open tag can also work completely fine.
8. Nesting in XML should be done correctly, it has a big importance in XML syntax. Conversely, HTML does not care much about nesting.

XML Rules

Case sensitivity

XML is case sensitive so start and end tag names must match exactly. For example, the following start and end tags have a mismatch in case and so are not well-formed,

```
<CARS>  
</Cars>
```

The end-tag name needs to have all capital letters, i.e. </CARS>, in order for it to match the start tag.

Syntax Rules for XML declaration

- The XML declaration is case sensitive and must begin with "<?xml>" where "xml" is written in lower-case.
- If document contains XML declaration, then it strictly needs to be the first statement of the XML document.

```
<?xml version="1.0" encoding="UTF-8"?>
```

Syntax Rules for Tags and Elements

- An XML document can have only one root element.

Wrong

```
<x>...</x>
<y>...</y>
```

Correct

```
<root>
<x>...</x>
<y>...</y>
</root>
```

- Each XML-element needs to be closed either with start or with end elements as shown below:

```
<element>....</element>
```

or in simple-cases, just this way:

```
<element/>
```

Syntax Rules for XML Attributes

- Attribute names are defined without quotation marks, whereas attribute values must always appear in quotation marks.

```
<a b="x">....</a>
```

- Same attribute cannot have two values in a syntax

Wrong

```
<a b="x" c="y" b="z">....</a>
```

Entity Reference Rule

Isolated markup characters are not allowed in text. However, they may be specified via entity references. For example, the < is specified by the entity reference < and the > symbol is >.

Comments Rule

The syntax for writing comments in XML is similar to that of HTML. Two dashes in the middle of a comment are not allowed:

Correct

```
<!-- This is a comment -->
```

Wrong

```
<!-- This is an invalid -- comment -->
```

White-space is Preserved in XML

XML does not truncate multiple white-spaces (HTML truncates multiple white-spaces to one single white-space):

XML Elements

An XML element is the fundamental building block of an XML document. It is used to store data in a structured format and represents a unit of information. Elements can contain:

1. **Text Content:** The main data or value within the element.
2. **Attributes:** Additional metadata or properties about the element.
3. **Nested Elements:** Other elements inside it, forming a hierarchical structure.
4. **Empty Content:** An element without any data, often self-closed.

Each element is defined using a start tag (e.g., *<name>*) and an end tag (e.g., *</name>*), or in the case of an empty element, a self-closing tag (e.g., *<element/>*).

Example Element

```
<name>Ram Sharma</name>
```

Characteristics of XML Elements

- **Start and End Tags:** XML elements have a start tag (e.g., *<element>*) and an end tag (e.g., *</element>*). Both are mandatory unless the element is self-closing.

```
<name>Ram Sharma</name>
```

- **Hierarchy and Nesting:** XML supports nested elements, creating a tree-like structure that represents relationships between data.

```
<person>
  <name>Ram Sharma</name>
  <age>30</age>
  <city>Patan</city>
</person>
```

- **Attributes:** Elements can have attributes that store additional metadata in key-value pairs (e.g., *<element attribute="value">*).

```
<book id="101" genre="fiction">
  <title>The Alchemist</title>
  <author>Paulo Coelho</author>
</book>
```

- **Self-Closing Tags:** If an element contains no data, it can be closed in the same tag using a slash (e.g., *<element/>*). An empty element is closed with a slash (/) and may have multiple attributes.

```

```

- **Case Sensitivity:** XML is case-sensitive, so *<Name>* and *<name>* are different.

Best Practices for XML Elements

- Use descriptive and meaningful element names to enhance readability and clarity.
- Avoid using special characters or spaces in element names; stick to alphanumeric characters and underscores.
- Consistently follow a naming convention (e.g., camelCase or snake_case) throughout your XML document. (Snake casing (or snake_case) is a naming convention in programming where words are written in lowercase and separated by underscores (_).)
- Use attributes for metadata sparingly and rely on nested elements for complex data.
- Properly indent nested elements to improve readability and maintainability.

XML Attributes

An XML attribute is used to provide additional metadata about an element. Attributes allow you to define the properties of a component in a more precise and structured manner. They are always included within the opening tag of an element.

```
<element_name  
  attribute1="value1"  
  attribute2="value2">  
  Contents ...</element_name>
```

Here, element_name is the name of the element. attribute1, attribute2 are attributes with unique names. value1, value2 are corresponding values of the attributes enclosed in single (') or double (") quotes.

Why use Attributes in XML?

- Attributes provide additional metadata, making XML elements more descriptive and informative.
- They help categorize elements, enabling better organization and grouping within the document.
- Attributes eliminate the need for creating new elements by distinguishing between elements of the same name.
- They simplify the representation of properties by using compact name-value pairs within the element's start tag.
- Attributes enhance data structure and facilitate easier parsing and processing in applications.

XML Elements vs. Attributes:

There are no rules about when to use attributes and when to use elements. Attributes are handy in HTML. But in XML they are difficult to read and maintain. It is better to use elements for data and to use attributes for information that is not relevant to the data.

In XML it is recommended to avoid attributes. Use elements instead.

Take a look at these examples:

First Example

```
<?xml version="1.0" encoding="UTF-8"?>  
<person gender="male">  
  <firstname>Ruskin</firstname>  
  <lastname>Bond</lastname>  
</person>
```

Second Example

```
<?xml version="1.0" encoding="UTF-8"?>  
<person>  
  <gender>male</gender>  
  <firstname>Ruskin</firstname>
```

```
        <lastname>Bond</lastname>
    </person>
```

In the first example gender is an attribute. In the second, gender is an element. Both examples provide the same information.

The following three XML documents contain exactly the same information:

A date attribute is used in the first example:

```
<?xml version="1.0" encoding="UTF-8"?>
<note date="10/01/2010">
    <to>Ram</to>
    <from>Shyam</from>
    <heading>Reminder</heading>
    <body>On Sunday we will watch Avengers!</body>
</note>
```

A date element is used in the second example:

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
    <date>10/01/2010</date>
    <to>Ram</to>
    <from>Shyam</from>
    <heading>Reminder</heading>
    <body>On Sunday We will watch Avengers!</body>
</note>
```

An expanded date element is used in the third:

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
    <date>
        <day>10</day>
        <month>01</month>
        <year>2010</year>
    </date>
    <to>Ram</to>
    <from>Shyam</from>
    <heading>Reminder</heading>
    <body>On Sunday We will watch Avengers!</body>
</note>
```


Some of the problems with using attributes are:

- attributes cannot contain multiple values (elements can)
- attributes cannot contain tree structures (elements can)
- attributes are not easily expandable (for future changes)

XML Attributes for Metadata:

The ID references assigned to the elements are used to identify XML elements. It is similar to the id attribute in HTML.

```
<messages>
  <note id="100">
    <to>Sapna</to>
    <from>Tom</from>
    <heading>Meeting</heading>
    Monday at 11 AM!
  </note>
  <note id="200">
    <to>Sapna</to>
    <from>Tom</from>
    <heading>Re: Meeting</heading>
    Thanks for the reminder!
  </note>
</messages>
```

In above example, For identifying the different notes, we use the id attributes, but they are not a part of the note itself. The metadata i.e., the data about data is to be stored as attributes. However, the data itself should be stored as elements.

XML Tree Structure

XML (eXtensible Markup Language) documents are structured as trees. This means that the data is organized hierarchically, with a single root element that contains child elements, which in turn can contain their own child elements, and so on. This hierarchical structure is often referred to as an "XML tree."

By using tree structure, you can get to know all succeeding branches and sub-branches starting from the root. The parsing starts at the root, then moves down the first branch to an element, take the first branch from there, and so on to the leaf nodes.

Example

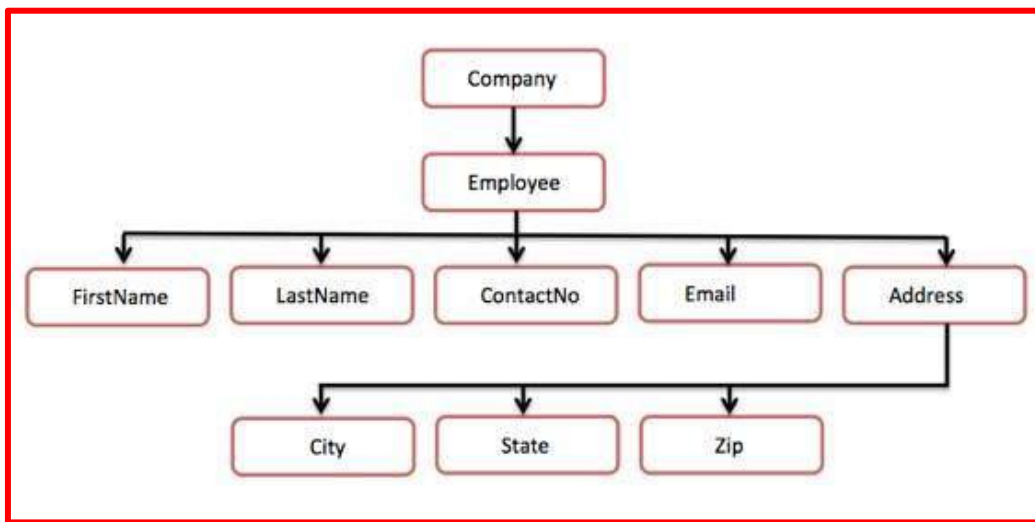
```
<?xml version = "1.0"?>
<Company>
  <Employee>
```

```

<FirstName>Tanmay</FirstName>
<LastName>Patil</LastName>
<ContactNo>1234567890</ContactNo>
<Email>tanmaypatil@xyz.com</Email>
<Address>
  <City>Bangalore</City>
  <State>Karnataka</State>
  <Zip>560212</Zip>
</Address>
</Employee>
</Company>

```

For the given XML, it is represented in tree form as following.



Example 2:

```

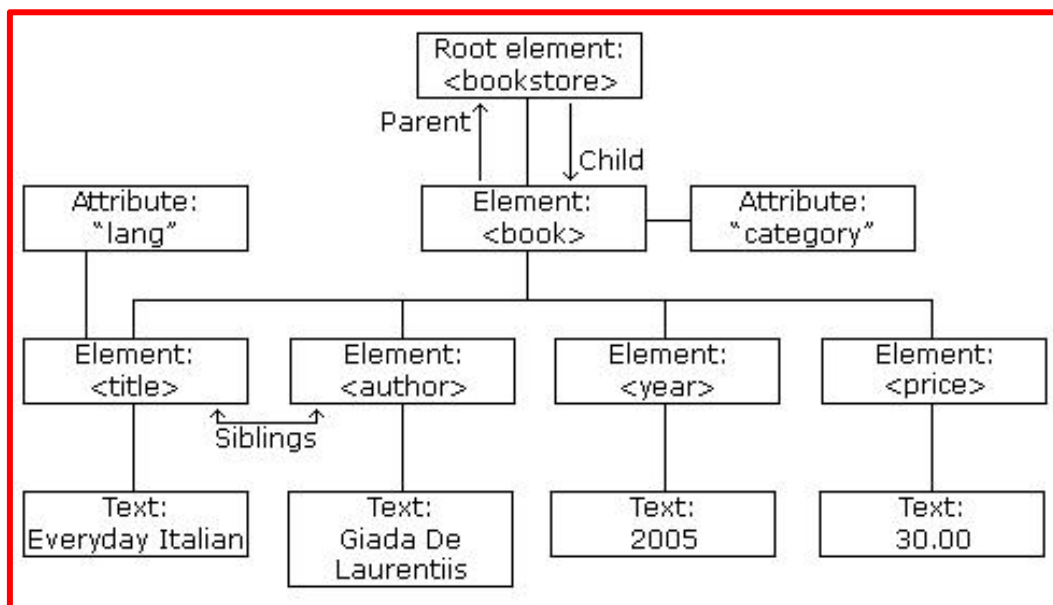
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
  </book>
</bookstore>

```

```

    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>

```



XML Namespace

Namespace is a mechanisms by which element and attribute name can be assigned to a group. The Namespace is identified by URI(Uniform Resource Identifiers).

Since XML allows designers to chose their own tag names, it is possible that two or more designers may choose the same tag names for some or all of their elements. XML namespace solves this problem. It provides a way to distinguish between XML elements that have the same local name but are, in fact, from different vocabularies. This is done by associating an element with a namespace. A namespace acts as a scope for all elements associated with it.

Example for name conflict:

Imagine you have an XML document that combines information about books and employees. Both books and employees have a <title> element, but they mean different things:

For a book, <title> refers to the book's name (e.g., "The Great Gatsby").

For an employee, <title> refers to their job title (e.g., "Manager").

Without namespaces, the XML might look like this:

```
<?xml version="1.0" ?>
<data>
  <book>
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
  </book>
  <employee>
    <name>John Doe</name>
    <title>Manager</title>
  </employee>
</data>
```

Here, the <title> element is used in two different contexts, which can cause confusion or errors when processing the XML.

To resolve this, we can define two namespaces:

1. One for book-related elements.
2. Another for employee-related elements.

Namespace Syntax

A Namespace is declared using reserved attributes. Such an attribute name must either be **xmlns** or begin with **xmlns:** shown as below –

```
<element xmlns:name = "URL">
```

Example

```
<?xml version="1.0" ?>
<data xmlns:bk="http://www.example.com/books"
      xmlns:emp="http://www.example.com/employees">
  <bk:book>
    <bk:title>The Great Gatsby</bk:title>
    <bk:author>F. Scott Fitzgerald</bk:author>
  </bk:book>
  <emp:employee>
    <emp:name>John Doe</emp:name>
    <emp:title>Manager</emp:title>
  </emp:employee>
</data>
```

Alternatively namespaces can be declared individually but its less common approach.

```
<?xml version="1.0" ?>
<data>
  <bk:book xmlns:bk="http://www.example.com/books">
    <bk:title>The Great Gatsby</bk:title>
    <bk:author>F. Scott Fitzgerald</bk:author>
  </bk:book>
  <emp:employee xmlns:emp="http://www.example.com/employees">
    <emp:name>John Doe</emp:name>
    <emp:title>Manager</emp:title>
  </emp:employee>
</data>
```

When a namespace is defined for an element, all child elements with the same prefix are associated with the same namespace.

XML Schema Languages

XML schema languages are used to define the structure, content, and semantics of XML documents. They provide a way to describe the elements, attributes, data types, and relationships within an XML document, ensuring that the document confirm to a specific format or set of rules.

Advantages of Schemas

- Define the characteristics and syntax of a set of documents.
- Independent groups can have a common format for interchanging XML documents.
- Software applications that process the XML documents know what to expect if they adhere to a formal schema.
- XML documents can be validated to verify that they conform to a given schema.
- Validation can be used as a debugging tool, directing the designer to items in a document that violate the schema.
- A schema can act as documentation for users defining or reading some set of XML documents.
- A schema can increase the reliability, consistency, and accuracy of exchanged documents.

There are multiple schema languages, among them [Document Type Definition \(DTD\)](#) and [XML Schema Definition \(XSD\)](#) are most commonly used schema languages.

Document Type Definition (DTD)

The XML Document Type Definition, commonly known as DTD, is a way to describe XML language precisely. DTDs check vocabulary and validity of the structure of XML documents against grammatical rules of appropriate XML language.

DTD contains a set of rules that control the structure and elements of XML files. When any XML file refers DTD file, it validates against those rules. It serves as a formal specification that outlines the elements, attributes, and their relationships within an XML document. They specify what elements can appear, their order, which attributes they can have, and what data types those attributes can contain.

An XML DTD can be either specified inside the document, or it can be kept in a separate document and then linked separately.

- **Internal DTD:** Inside the document
- **External DTD:** Specified in a separate document and linked later.

DTD Syntax

```
<!DOCTYPE element DTD identifier  
[  
    declaration1  
    declaration2  
    .....  
>
```

In the above syntax,

- The **DTD** starts with **<!DOCTYPE** delimiter.
- An **element** tells the parser to parse the document from the specified root element.
- **DTD identifier** is an identifier for the document type definition, which may be the path to a file on the system or URL to a file on the internet. If the DTD is pointing to external path, it is called **External Subset**.
- **The square brackets []** enclose an optional list of entity declarations called *Internal Subset*.

Example

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE bookstore [  
    <!ELEMENT bookstore (book+)>  
    <!ELEMENT book (title, author, year, price)>  
    <!ATTLIST book category CDATA #REQUIRED>  
    <!ELEMENT title (#PCDATA)>  
    <!ATTLIST title lang CDATA #REQUIRED>  
    <!ELEMENT author (#PCDATA)>  
    <!ELEMENT year (#PCDATA)>  
    <!ELEMENT price (#PCDATA)>
```

```
]>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```

DTD - Elements

A DTD element is declared with an ELEMENT declaration. When an XML file is validated by DTD, parser initially checks for the root element and then the child elements are validated.

Syntax:

```
<!ELEMENT elementname (content)>
```

ELEMENT declaration is used to indicate the parser that you are about to define an element.

elementname is the element name (also called the *generic identifier*) that you are defining.

content defines what content (if any) can go within the element.

Content of elements declaration in a DTD can be categorized as below:

- Elements with empty content
- Elements with only character data
- Elements with children elements
- Elements with mixed contents
- Any content

Empty Content:

In XML, empty content is used to represent elements that do not contain any text or child elements but may still carry information through attributes. Empty elements are self-contained and are often used for specific purposes, such as marking a position, providing metadata, or defining a placeholder.

```
<!ELEMENT elementname EMPTY >

DTD
<!ELEMENT image EMPTY>

XML
<image src="photo.jpg" alt="A beautiful landscape"/>
```

Elements with only character data:

In XML, **elements with character data** (also known as **text content**) are used to store and represent textual information. These elements contain **parsed character data (PCDATA)**, which means the text is processed by the XML parser to resolve entities and handle special characters or **character data (CDATA)** which are used to include blocks of text that should not be parsed by the XML parser. Elements with character data are one of the most common types of elements in XML documents.

NOTE: Though CDATA can be used for character data, but are not oftenly used since the contents are not parsed by xml parser.

```
<!ELEMENT elementName (#PCDATA)>

DTD
<!ELEMENT author (#PCDATA)>

XML
<author>J K. Rowling</author>
```

Elements with children elements

In element declaration with element content, the content would be allowable elements within parentheses. We can also include more than one element and each element must have its own definition within the DTD.

When children are declared in a sequence separated by commas, the children must appear in the same sequence in the document. In a full declaration, the children must also be declared, and the children can also have children.

```
<!ELEMENT elementname (child1, child2...)>

DTD
<!ELEMENT book (title, author, year, price)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT price (#PCDATA)>

XML
<book>
```



```

<title>Harry Potter</title>
<author>J K. Rowling</author>
<year>2005</year>
<price>29.99</price>
</book>

```

Suppose you need to allow one element or another, but not both. In such cases you must use the pipe (|) character. The pipe functions as an **exclusive OR**.

```
<!ELEMENT elementname (child1 | child2...)>
```

DTD

```
<!ELEMENT phone (mobile | landline)>
```

XML

```
<phone>
```

```
  <mobile>123-456-7890</mobile>
```

```
</phone>
```

Or

```
<phone>
```

```
  <landline>555-1234</landline>
```

```
</phone>
```

List of operators and syntax rules which can be applied in defining child elements

Operator	Syntax	Description	Example
+	<!ELEMENT element-name (child1+)>	It indicates that child element can occur one or more times inside parent element.	<!ELEMENT address (name+)> Child element <i>name</i> can occur one or more times inside the element name <i>address</i> .
*	<!ELEMENT element-name (child1*)>	It indicates that child element can occur zero or more times inside parent element.	<!ELEMENT address (name*)> Child element <i>name</i> can occur zero or more times inside the element name <i>address</i> .
?	<!ELEMENT element-name (child1?)>	It indicates that child element can occur zero or one time inside parent element.	<!ELEMENT address (name?)> Child element <i>name</i> can occur zero or one time inside the element name <i>address</i> .

,	<code><!ELEMENT element-name (child1, child2)></code>	It gives sequence of child elements separated by comma which must be included in the the element-name.	<code><!ELEMENT address (name, company)></code> Sequence of child elements <i>name, company</i> , which must occur in the same order inside the element name <i>address</i> .
	<code><!ELEMENT element-name (child1 child2)></code>	It allows making choices in the child element.	<code><!ELEMENT address (name company)></code> It allows you to choose either of child elements i.e. <i>name</i> or <i>company</i> , which must occur in inside the element name <i>address</i> .

Elements with mixed contents

This is the combination of (#PCDATA) and children elements. Within mixed content models, text can appear by itself or it can be in between the elements.

```
<!ELEMENT elementname (#PCDATA|child1|child2)*>

DTD
<!ELEMENT paragraph (#PCDATA|bold|italic)*>
<!ELEMENT bold (#PCDATA)>
<!ELEMENT italic (#PCDATA)>

XML
<paragraph>
  This is a <bold>bold</bold> statement and this is
  <italic>italic</italic>.
  You can also have <bold><italic>bold and italic</italic></bold>
  text.
</paragraph>
```

The * (asterisk) indicates that the content (text or child elements) can appear zero or more times and we can use operators from above table or not add that as well.

Elements with any content

You can declare an element using the ANY keyword in the content. It is most often referred to as mixed category element. ANY is useful when you have yet to decide the allowable contents of the element.

```
<!ELEMENT elementname ANY>
```

```

DTD
<!ELEMENT address ANY>

XML
<address>
  Here's a bit of sample text
</address>

```

DTD Attributes

These are used to provide the additional information along with elements. In DTD, the attributes are declared by using "ATTLIST" Keyword.

Syntax

```
<!ATTLIST element-name attribute-name attribute-type attribute-value>
```

where the attribute-type can be one of the following

Type	Description
CDATA	The value is character data
(en1 en2 ...)	The value must be one from an enumerated list
ID	The value is a unique id
IDREF	The value is the id of another element
IDREFS	The value is a list of other ids
	The value is a valid XML name. It is a string that can contain:
NMTOKEN	• Letters (A-Z, a-z) • Digits (0-9) • Periods (.) • Hyphens (-) • Underscores (_)
NMTOKENS	The value is a list of valid XML names.
ENTITY	The value is an entity
ENTITIES	The value is a list of entities
NOTATION	The value is a name of a notation
xml:	The value is a predefined xml value

and the attribute-value can be one of the following

Value	Explanation
value	The default value of the attribute
#REQUIRED	The attribute is required
#IMPLIED	The attribute is optional
#FIXED value	The attribute value is fixed

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE person [
  <!ELEMENT person (name, age)>
  <!ATTLIST person
    gender      CDATA #REQUIRED
    nationality  CDATA #IMPLIED
    status       CDATA #FIXED "single"
    id          ID #REQUIRED
  >
]>
<person gender="male" nationality="American" id="12345">
  <name>John Doe</name>
  <age>30</age>
</person>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE person [
  <!ELEMENT person (name, age, address)>
  <!ATTLIST person
    id          ID #REQUIRED
    friend       IDREF #IMPLIED
    contacts     IDREFS #IMPLIED
    username     NMTOKEN #REQUIRED
    roles        NMTOKENS #IMPLIED
    picture      ENTITY #IMPLIED
  >
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT age (#PCDATA)>
  <!ELEMENT address (street, city)>
  <!ATTLIST address
    id          ID #REQUIRED
    country     NMTOKEN #REQUIRED
  >
  <!ELEMENT street (#PCDATA)>
  <!ELEMENT city (#PCDATA)>
```

```

]>
<!-- Person 1 -->
<person id="P123" friend="P456" contacts="P789 P456"
username="john_doe" roles="admin user" picture="profile_picture">
  <name>John Doe</name>
  <age>30</age>
  <address id="A123" country="USA">
    <street>Main Street 123</street>
    <city>New York</city>
  </address>
</person>

<!-- Person 2 -->
<person id="P456" friend="P123" contacts="P789" username="jane_doe"
roles="user" picture="profile_picture">
  <name>Jane Doe</name>
  <age>28</age>
  <address id="A456" country="USA">
    <street>Park Avenue 456</street>
    <city>Los Angeles</city>
  </address>
</person>

<!-- Person 3 -->
<person id="P789" username="mark_smith" roles="user"
picture="profile_picture">
  <name>Mark Smith</name>
  <age>35</age>
  <address id="A789" country="USA">
    <street>Sunset Blvd 789</street>
    <city>Chicago</city>
  </address>
</person>

```

DTD Entities

Entities are used to define shortcuts to special characters. Entities can be declared internal or external. It has 3 parts.

1. An ampersand (&)
2. An entity name
3. A semicolon (;)

Syntax:

```
<!ENTITY entity-name "entity-value">
```

Example Internal:

```
<!ENTITY writer "Tom Nolan">
<!ENTITY copyright "Copyright TutorialReference.">

XML
<author>&writer;&copyright;</author>
```

Example External:

```
<!ENTITY entity-name SYSTEM "URI">
or
<!ENTITY entity-name SYSTEM "URL">
```

```
DTD
<!ENTITY writer SYSTEM "https://tutorialreference.com/entities.dtd">
<!ENTITY copyright SYSTEM
"https://tutorialreference.com/entities.dtd">

XML
<author>&writer;&copyright;</author>
```

Internal DTD:

An Internal DTD (Document Type Definition) is a collection of rules and declarations that are included directly within an XML document. It specifies the structure, elements, and attributes that the document must follow to conform to a particular format. This DTD is incorporated within the `<!DOCTYPE>` declaration in the XML file itself.

Syntax

```
<!DOCTYPE root-element [element-declarations]>
```

Here *root-element* is the name of root element and *element-declarations* is where you declare the elements.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookstore [
  <!ELEMENT bookstore (book+)>
  <!ELEMENT book (title, author, year, price)>
  <!ATTLIST book category CDATA #REQUIRED>
  <!ELEMENT title (#PCDATA)>
  <!ATTLIST title lang CDATA #REQUIRED>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT year (#PCDATA)>
```

```

    <!--ELEMENT price (#PCDATA)-->
  ]>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>

```

External DTD

An External DTD (Document Type Definition) is a way to separate the DTD from the XML document itself. Instead of defining the structure and rules of the XML file within the XML itself (internal DTD), you link to a DTD stored in an external file. This approach helps maintain a clean separation between the XML content and its validation rules, making it easier to reuse the same DTD across multiple XML documents.

Syntax

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root-element SYSTEM "path-to-your-dtd-file.dtd">
<root-element>
  <!-- Your XML content here -->
</root-element>

```

Example

Book.dtd

```

<!--ELEMENT bookstore (book+)-->
<!--ELEMENT book (title, author, year, price)-->
<!--ATTLIST book category CDATA #REQUIRED-->
<!--ELEMENT title (#PCDATA)-->
<!--ATTLIST title lang CDATA #REQUIRED-->
<!--ELEMENT author (#PCDATA)-->
<!--ELEMENT year (#PCDATA)-->

```

```
<!ELEMENT price (#PCDATA)>
```

Book.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookstore SYSTEM "Book.dtd">
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```

XML Schema Definition (XSD)

An XML schema, commonly known as an XML Schema Definition (XSD), formally describes what a given XML document can contain, in the same way that a database schema describes the data that can be contained in a database (i.e. table structure, data types, constraints etc.). The XML schema defines the shape, or structure, of an XML document, along with rules for data content and semantics such as what fields an element can contain, which sub elements it can contain and how many items can be present. It can also describe the type and values that can be placed into each element or attribute. The XML data constraints are called facets and include rules such as min and max length.

- XML Schema Definition (XSD) is currently the de facto standard for describing XML documents and is the XML Schema standard. XSD is controlled by the World Wide Web Consortium (W3C). An XSD is itself an XML document, and there is even an XSD to describe the XSD standard.
- Document Type Definition (DTD) was the first formalized standard but has now, in most cases, been superseded by XSD.
- XML Data Reduced (XDR) was an early attempt but Microsoft to provide a more comprehensive standard than DTD. This standard has been phased out in the Microsoft products in favour of XSD.

Example XSD

Book.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.a.com"
xmlns="http://www.a.com"
elementFormDefault="qualified">

  <xs:element name="bookstore">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="title" type="xs:string" />
              <xs:attribute name="lang" type="xs:string"
use="required"/>
            </xs:sequence>
            <xs:element name="author" type="xs:string"/>
            <xs:element name="year" type="xs:integer"/>
            <xs:element name="price" type="xs:decimal"/>
            </xs:sequence>
            <xs:attribute name="category" type="xs:string"
use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Book.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore xmlns="http://www.a.com"
xsi:schemaLocation="http://www.a.com book.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema" >
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
  </book>
</bookstore>
```

```

    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>

```

The <schema> element

The <schema> element is the root element of every XML Schema.

```

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.a.com"
  xmlns="http://www.a.com"
  elementFormDefault="qualified">
  .
  .
  .
</xs:schema>

```

Here,

- **xmlns:xs="http://www.w3.org/2001/XMLSchema"** indicates that the elements and data types used in the schema come from the <http://www.w3.org/2001/XMLSchema> namespace. It also specifies that the elements and data types that come from the <http://www.w3.org/2001/XMLSchema> namespace should be prefixed with :xs.
- **targetNamespace="https://www.a.com"** indicates that the elements defined by this schema (book, title, author, year, price.) come from the <https://www.a.com> namespace.
- **xmlns="https://www.a.com"** indicates that the default namespace is <https://www.a.com>.
- **elementFormDefault="qualified"** indicates that any elements used by the XML instance document which were declared in this schema must be elements and attributes are in the targetNamespace of the schema.

Referencing a Schema in an XML Document

```

<bookstore
  xmlns="https://www.a.com"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://www.a.com book.xsd">
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
  </book>
</bookstore>

```

```

    <price>29.99</price>
  </book>
</bookstore>

```

- `xmlns="https://www.a.com"` specifies the default namespace declaration. This declaration tells to the schema-validator that all the elements used in this XML document are declared in the `https://www.a.com` namespace.
- `xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"` declares a standard namespace prefix (xsi) for a core namespace used in the XML schema
- `xsi:schemaLocation="https://www.a.com book.xsd"` The `schemaLocation` attribute has two values separated by a space: the first is the name space to use while the second value is the location of the XML schema to use for that namespace.

XSD Simple Elements

In a XML Schema, a simple element is an XML element that contains only text. *It can't contain any other elements or attributes.*

Only text is misleading: the text can be of many different types. It can be one of the types included in the XML Schema definition or it can be a custom type that you can define yourself. You can also add restrictions (called facets) to a data type in order to limit its content, or you can require the data to match a specific pattern.

Syntax

```
<xs:element name="my-name" type="my-type"/>
```

- `my-name` is the name of the element
- `my-type` is the data type of the element.

XML Schema has a lot of built-in data types. The most common type are:

- `xs:string`
- `xs:decimal`
- `xs:integer`
- `xs:boolean`
- `xs:date`
- `xs:time`
- `xs:anyuri`

Example

```

<xs:element name="author" type="xs:string"/>
<xs:element name="year" type="xs:integer"/>
<xs:element name="price" type="xs:decimal"/>

```

XML

```
<author>J K. Rowling</author>
```

```
<year>2005</year>
<price>29.99</price>
```

Default and Fixed Values for Simple Elements

Simple elements can have a **default value** or a **fixed value** specified.

A **default value** is automatically assigned to the element when no other value is specified. For example `default="red"`.

A **fixed value** is also automatically assigned to the element, and you can't specify another value. For example `fixed="red"`.

```
<xs:element name="color" type="xs:string" default="red"/>
<xs:element name="color" type="xs:string" fixed="red"/>
```

XML, default

```
<color></color>      (color value is red)
```

```
<color>blue</color>  (color value is blue)
```

Fixed

```
<color>red</color>
```

```
<color>blue</color> (Invalid)
```

XSD Attributes

In XML Schema Definition (XSD), an attribute defines additional information about an element. It provides metadata or supplementary data for the element. Attributes are part of an element's structure but are not the main content (which is typically the text within the element itself).

Syntax

```
<xs:attribute name="my-name" type="my-type"/>
```

- my-name is the name of the attribute.
- my-type is the data type of the attribute.

The most common type are:

- xs:string
- xs:decimal
- xs:integer
- xs:boolean
- xs:date
- xs:time
- xs:anyuri

```

SCHEMA
<xs:element name="title">
  <xs:complexType>
    <xs:attribute name="lang" type="xs:string"/>
  </xs:complexType>
</xs:element>

XML
<title lang="en">Harry Potter</title>

```

OR

(Can skip for exam)

```

SCHEMA
<xs:element name="title">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="lang" type="xs:string"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>

```

OR

```

SCHEMA
<xs:element name="title" type="xs:string">
  <xs:attribute name="lang" type="xs:string"/>
</xs:element>

XML
<title lang="en">Harry Potter</title>

```

NOTE: An element that has attributes, is already by definition a complex type. However, you do not have to explicitly define the complex type. the xsd processor will implicitly create a complex type for your element.

Default and Fixed Values for Attributes

Attributes can have a **default value** or a **fixed value** specified.

A **default value** is automatically assigned to the attribute when no other value is specified. For example lang="en".

A **fixed value** is also automatically assigned to the attribute, and you can't specify another value. For example lang="en".

```
<xs:attribute name="lang" type="xs:string" default="en"/>
<xs:attribute name="lang" type="xs:string" fixed="en"/>
```

Optional and Required Attributes

All attributes are optional by default. To specify that an attribute is required, just use the use attribute:

```
<xs:attribute name="lang" type="xs:string" use="required"/>
```

XSD Complex Elements

A complex element is an XML element that contains other elements and/or attributes.

Complex elements can be of four types:

- empty elements
- elements that contain only other elements
- elements that contain only text
- elements that contain both other elements and text

[Note: Each of these elements may contain attributes as well!](#)

empty elements

To define a type with no content, we must define a type that allows elements in its content, but we do not actually declare any elements, like this:

```
SCHEMA
<xs:element name="product">
  <xs:complexType>
    <xs:attribute name="itemid" type="xs:positiveInteger"/>
  </xs:complexType>
</xs:element>

XML
<product itemid="1234"/>
```

Or you can give the complexType element a name, and let the "product" element have a type attribute that refers to the name of the complexType. In this way, several elements can refer to the same complex type:

```
<xs:element name="product" type="prodtype"/>

<xs:complexType name="prodtype">
  <xs:attribute name="itemid" type="xs:positiveInteger"/>
</xs:complexType>
```

Complex Types Containing Elements Only

An XML element, "book", that contains only other elements:

```
<book>
  <title>A Great Book</title>
  <author>Tom Nolan</author>
  <publisher>Tutorial Reference</publisher>
</book>
```

You can define the "book" element in a schema, like this:

```
<xs:element name="book">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="author" type="xs:string"/>
      <xs:element name="publisher" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

NOTE:

The `<xs:sequence>` tag means that elements defined ("title", "author", "publisher") must appear in that order inside a "book" element.

Or you can give the complexType element a name, and let the "book" element have a type attribute that refers to the name of the complexType **(Can skip for exam)**

```
<xs:element name="book" type="bookinfo"/>

<xs:complexType name="bookinfo">
  <xs:sequence>
    <xs:element name="title" type="xs:string"/>
    <xs:element name="author" type="xs:string"/>
    <xs:element name="publisher" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Complex types that contain only text

An XML element, "book", that contains only text:

```
<book genre="fiction" isbn="113355">The Great Gatsby</book>
```

We can declare Complex Text-only elements using the following methods:

```
<xs:element name="book">
  <xs:complexType>
```

```

    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="genre" type="xs:string"/>
        <xs:attribute name="isbn" type="xs:string"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>

```

Or

(Can skip for exam)

```

<xs:element name="book" type="bookinfo"/>

<xs:complexType name="bookinfo">
  <xs:simpleContent>
    <xs:extension base="xs:string">
      <xs:attribute name="genre" type="xs:string"/>
      <xs:attribute name="isbn" type="xs:string"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>

```

Complex types that contain both element and text (Mixed Element)

A mixed complex type element can contain attributes, elements, and text.

Consider this example:

```

<mail>
  Dear Mr. <name>John Smith</name>.
  Your order <orderid>1234</orderid>
  will be delivered on <deliverydate>2001-07-13</deliverydate>.
</mail>

```

Define complexType with attribute **mixed set to true**. mixed attribute allow to have character data between elements.

```

<xs:element name="mail">
  <xs:complexType mixed="true">
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="orderid" type="xs:positiveInteger"/>
      <xs:element name="deliverydate" type="xs:date"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```

Or

(Can skip for exam)

```
<xs:element name="mail" type="mailinfo"/>

<xs:complexType name="mailinfo" mixed="true">
  <xs:sequence>
    <xs:element name="name" type="xs:string"/>
    <xs:element name="orderid" type="xs:positiveInteger"/>
    <xs:element name="deliverydate" type="xs:date"/>
  </xs:sequence>
</xs:complexType>
```

XSD Restrictions

XML Restrictions are used to define acceptable values for XML elements or attributes. Restrictions on XML elements are called facets.

Restrictions for Datatypes

The following table lists the restrictions for different data types:

Constraint	Description
<i>enumeration</i>	Defines a list of acceptable values
<i>fractionDigits</i>	Specifies the maximum number of decimal places allowed. Must be equal to or greater than zero
<i>length</i>	Specifies the exact number of characters or list items allowed. Must be equal to or greater than zero
<i>maxExclusive</i>	Specifies the upper bounds for numeric values (the value must be less than this value)
<i>maxInclusive</i>	Specifies the upper bounds for numeric values (the value must be less than or equal to this value)
<i>maxLength</i>	Specifies the maximum number of characters or list items allowed. Must be equal to or greater than zero
<i>minExclusive</i>	Specifies the lower bounds for numeric values (the value must be greater than this value)
<i>minInclusive</i>	Specifies the lower bounds for numeric values (the value must be greater than or equal to this value)
<i>minLength</i>	Specifies the minimum number of characters or list items allowed. Must be equal to or greater than zero
<i>pattern</i>	Defines the exact sequence of characters that are acceptable

Constraint	Description
<i>totalDigits</i>	Specifies the exact number of digits allowed. Must be greater than zero
<i>whiteSpace</i>	Specifies how white space (line feeds, tabs, spaces, and carriage returns) is handled

Restrictions on Values

Example of restrictions on values:

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Restrictions on a Set of Values

The enumeration constraint is used to limit the content of an XML element to a set of acceptable values.

Example of enumeration constraint on car element where only acceptable values are "Audi", "Golf", "BMW":

```
<xs:element name="car">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="Audi"/>
      <xs:enumeration value="Golf"/>
      <xs:enumeration value="BMW"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

An equivalent way to write the above example is as follows: (Can skip for exam)

```
<xs:element name="car" type="carType"/>

<xs:simpleType name="carType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Audi"/>
    <xs:enumeration value="Golf"/>
    <xs:enumeration value="BMW"/>
  </xs:restriction>
</xs:simpleType>
```

note

In this case the type `carType` can be used by other elements because it is not a part of the `car` element.

Restrictions on a Series of Values

The pattern constraint is used to limit the content of an XML element to define a series of numbers or letters that can be used.

Example of pattern constraint on letter element where only acceptable value is one of the lowercase letters from a to z:

```
<xs:element name="letter">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-z]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on initials element where only acceptable value is three of the uppercase letters from a to z:

```
<xs:element name="initials">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[A-Z][A-Z][A-Z]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on initials element where only acceptable value is three of the lowercase or uppercase letters from a to z:

```
<xs:element name="initials">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z][a-zA-Z][a-zA-Z]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on choice element where only acceptable value is one of the following letters a, b or c:

```
<xs:element name="choice">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[abc]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on prodid element where only acceptable value is five digits in a sequence, and each digit must be in a range from 0 to 9:

```
<xs:element name="prodid">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:pattern value="[0-9][0-9][0-9][0-9][0-9]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on letter element where only acceptable value is zero or more occurrences of lowercase letters from a to z:

```
<xs:element name="letter">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="([a-z])*"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on letter element where only acceptable value is one or more pairs of letters, consisting of a lower case letter followed by an uppercase letter.

```
<xs:element name="letter">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="([a-z][A-Z])+"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on gender element where only acceptable value is male or female:

```
<xs:element name="gender">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="male|female"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of pattern constraint on password element where there must be exactly eight characters in a row and those characters must be lowercase or uppercase letters from a to z, or a number from 0 to 9:

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
```

```

        <xs:pattern value="[a-zA-Z0-9]{8}"/>
    </xs:restriction>
</xs:simpleType>
</xs:element>

```

Restrictions on Whitespace Characters

The whitespace constraint is used to specify how whitespace characters should be handled.

Example of whitespace constraint on address element where the whitespace constraint is set to preserve, which means that the XML processor will not remove any white space characters:

```

<xs:element name="address">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:whitespace value="preserve"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>

```

Example of whitespace constraint on address element where the whitespace constraint is set to replace, which means that the XML processor will replace all white space characters (line feeds, tabs, spaces, and carriage returns) with spaces:

```

<xs:element name="address">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:whitespace value="replace"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>

```

Example of whitespace constraint on address element where the whitespace constraint is set to collapse, which means that the XML processor will remove all white space characters (line feeds, tabs, spaces, carriage returns are replaced with spaces, leading and trailing spaces are removed, and multiple spaces are reduced to a single space):

```

<xs:element name="address">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:whitespace value="collapse"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>

```

Restrictions on Length

The length, maxLength and minLength constraints are used to limit the length of a value in an element.

Example of length constraint on password element where the value must be exactly eight characters:

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:length value="8"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of maxLength and minLength constraints on password element where the value must be minimum five characters and maximum eight characters:

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:minLength value="5"/>
      <xs:maxLength value="8"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Example of minLength constraint on value field of title element which has attribute as well

```
<xs:element name="title">max
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:minLength value="6"/>
          </xs:restriction>
        </xs:simpleType>
        <xs:attribute name="lang" type="xs:string"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

```
<book category="children">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <date>
    <year>2005</year>
    <time>
      <hour>10</hour>
      <minute>12</minute>
    </time>
  </date>
</book>
```

```
</date>
<price>29.99</price>
</book>
```

We can perform same restriction on attributes as well

```
<xs:attribute name="phoneNumber" type="xs:string">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="\d{3}-\d{3}-\d{4}"/>
    </xs:restriction>
  </xs:simpleType>
</xs:attribute>
```

XSL

XSL stands for **Extensible Stylesheet Language**. It is a style language for XML (just as CSS is a style language for HTML).

In XML documents tags are not predefined. World Wide Web Consortium (W3C) developed XSL to understand and to style an XML document, which can act as XML based Stylesheet Language.

XSL stands for Extensible Styles Language and is a very powerful language for applying styles to XML documents. XSL has two languages — a formatting language and a transformation language.

The formatting language allows you to apply styles similar to what CSS does. Browser support for the XSL formatting language is limited at this stage.

The transformation language is known as XSLT (XSL Transformations). XSLT allows you to transform your XML document into another form. For example, you could use XSLT to dynamically output some (or all) of the contents of your XML file into an HTML document containing other content.

Need for XSL

In case of HTML document, tags are predefined such as table, div, and span; and the browser knows how to add style to them and display those using CSS styles. But in case of XML documents, tags are not predefined. In order to understand and style an XML document, World Wide Web Consortium (W3C) developed XSL which can act as XML based Stylesheet Language. An XSL document specifies how a browser should render an XML document.

XSL consists of four parts:

- **XSLT (Extensible stylesheet language Transformation)**: a language for transforming XML documents
- **XPath**: - a language for navigating in XML documents
- **XSL-FO (Extensible stylesheet language Formatting Object)**: - a language for formatting XML documents (discontinued in 2013)
- **XQuery**: - a language for querying XML documents

XPATH

The **XML Path (XPath)** language is a query language for XML documents, developed by the W3 consortium.

XPath (XML Path Language) is a query language for selecting nodes from an XML document. It uses a path-like syntax to identify and navigate through elements and attributes within the XML structure.

It is a powerful tool for navigating XML documents. It allows you to locate elements and attributes based on a variety of criteria, such as element names, attribute values, or content. Understanding XPath is essential for tasks like web scraping, automated testing (e.g., Selenium), and data manipulation in XML formats.

Example XML

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book>
    <title lang="en">The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <price>10.99</price>
  </book>
  <book>
    <title lang="en">To Kill a Mockingbird</title>
    <author>Harper Lee</author>
    <price>7.99</price>
  </book>
  <book>
    <title lang="es">Cien años de soledad</title>
    <author>Gabriel García Márquez</author>
    <price>12.50</price>
  </book>
</bookstore>
```

Here are some basic XPath concepts and how they can be applied to the XML:

- **Basic Selection:** You can select elements directly using **/**, **//**, and element names.
- **Predicates:** Use **[]** to filter nodes based on conditions.
- **Attributes:** Use **@** to select elements based on attributes.
- **Logical Operators:** Combine conditions using **and**, **or**.
- **Text Content:** Use **text()** to select the content inside an element.
- **Axis Specifiers :** Use **.** and **..** for navigating through the XML document in the context of the current node.

Example & Description

Result

/bookstore	
Selects the root element.	<bookstore> (root element)
//book	
Selects nodes anywhere in the document.	All <book> elements: <book>, <book>, <book>
/bookstore/book	
Selects all <book> elements inside <bookstore>.	All <book> elements: <book>, <book>, <book>
/bookstore/book/title	
Selects the <title> element of all <book> elements inside <bookstore>.	All <title> elements: "The Great Gatsby", "To Kill a Mockingbird", "Cien años de soledad"
/bookstore/book[2]	
Selects the second <book> element.	The second <book> element: <book><title>To Kill a Mockingbird</title><author>Harper Lee</author><price>7.99</price></book>
/bookstore/book[author="Harper Lee"]	
Selects the <book> element where <author> is "Harper Lee".	<book><title>To Kill a Mockingbird</title><author>Harper Lee</author><price>7.99</price></book>
/bookstore/book[price=10.99]	
Selects the <book> element where the price is 10.99.	<book><title>The Great Gatsby</title><author>F. Scott Fitzgerald</author><price>10.99</price></book>
/bookstore/book/title[@lang="en"]	
Selects the <title> element where the lang attribute is "en".	"The Great Gatsby", "To Kill a Mockingbird"
/bookstore/book/title[@lang="es"]	
Selects the <title> element where the lang attribute is "es".	"Cien años de soledad"
/bookstore/book[price>10]	
	<book><title>Cien años de soledad</title><author>Gabriel García Márquez</author><price>12.50</price></book>

Selects books where the price is greater than 10.

/bookstore/book[author="F. Scott Fitzgerald" or author="Harper Lee"]	<code><book><title>The Great Gatsby</title><author>F. Scott Fitzgerald</author><price>10.99</price></book></code> , <code><book><title>To Kill a Mockingbird</title><author>Harper Lee</author><price>7.99</price></book></code>
Selects books by either "F. Scott Fitzgerald" or "Harper Lee".	

/bookstore/book/title[1]/text()	"The Great Gatsby"
Selects the text content of the first <title> element.	

/bookstore/book/title/text()	"The Great Gatsby", "To Kill a Mockingbird", "Cien años de soledad"
Selects the text content of all <title> elements.	

@lang	"en", "en", "es"
/bookstore/book/title/@lang	
Selects an attribute of an element.	

/bookstore/book/author/text()	"F. Scott Fitzgerald", "Harper Lee", "Gabriel García Márquez"
Selects the text content of an element.	

/bookstore/*	<code><book></code> (direct child of <code><bookstore></code>)
Selects all child elements of the current node.	

./book (from a context of <book>)	<code><book><title>To Kill a Mockingbird</title><author>Harper Lee</author><price>7.99</price></book></code>
Selects the current node.	

../bookstore (from a context of <book>)	<code><bookstore></code> (the parent of <code><book></code>)
Selects the parent of the current node.	

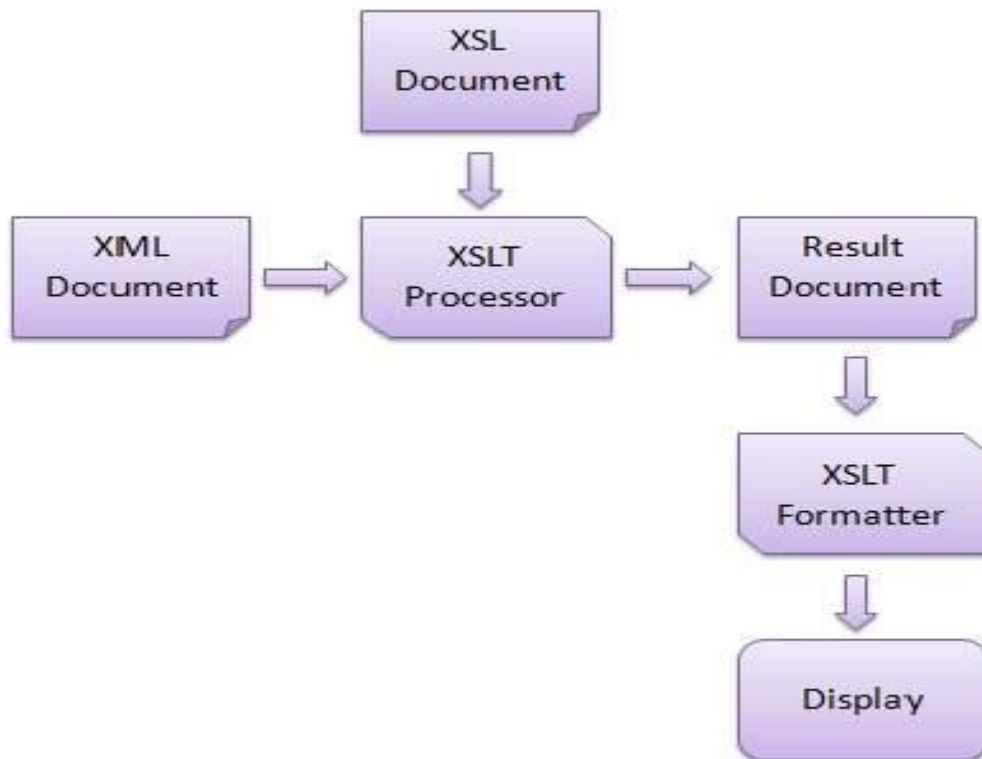
XSLT

XSLT stands for XSL Transformations and it is used to transform XML data from one format to another. For example, you could use XSLT to dynamically output some (or all) of the contents of your XML file into an HTML document containing other content. XSLT (eXtensible Stylesheet Language Transformations) provides the ability to transform XML data from one format to another automatically. With XSLT you can add/remove elements and attributes to or from the output file. You can also rearrange and sort elements, perform tests and make decisions about which elements to hide and display, and a lot more.

How does XSLT work

A common way to describe the transformation process is to say that XSLT transforms an XML source-tree into an XML result-tree.

The XSLT stylesheet is written in XML format. The XSLT processor takes the XSLT stylesheet and applies transformation rules on the target XML document and then generates a formatted document in XML, HTML or text format. This formatted document is then used by the XSLT formatter to generate the actual output that is to be displayed to the end user.



XSLT Syntax

The root element that declares the document to be an XSL style sheet is `<xsl:stylesheet>` or `<xsl:transform>`.

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

OR

```
<xsl:transform version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

`<xsl:stylesheet>` is essentially the same as `<xsl:transform>`, and can also be used as the root element of an XSLT stylesheet. Both `<xsl:transform>` and `<xsl:stylesheet>` are valid, but `<xsl:stylesheet>` is more commonly used.

Transformation steps

Step1: Prepare XML and create requirement for transformation.

Book.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```

Requirements:

1. Page should have a title Book Store.
2. Page should have a table of book details.
3. Columns should have following headers: title, author, year, price and category attribute of book element.
4. Table must contain details of the book store accordingly.

Step2: Create an XSL Style Sheet

Name it as "books.xsl" and save it in the same location of "books.xml".

```
<?xml version="1.0" encoding="UTF-8"?>

<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
```

```

<html>
  <body>
    <h2>The Book Store</h2>
    <table border="1">
      <tr bgcolor="#6565d5">
        <th>Title</th>
        <th>Author</th>
        <th>Year</th>
        <th>Price</th>
        <th>Category</th>
      </tr>
      <xsl:for-each select="bookstore/book">
        <tr>
          <td><xsl:value-of select="title"/></td>
          <td><xsl:value-of select="author"/></td>
          <td><xsl:value-of select="year"/></td>
          <td><xsl:value-of select="price"/></td>
          <td><xsl:value-of select="@category" /></td>
        </tr>
      </xsl:for-each>
    </table>
  </body>
</html>
</xsl:template>
</xsl:stylesheet>

```

Step3. Link the XSL Style Sheet to the XML Document

Add the XSL stylesheet reference to your XML document ("books.xml"):

```

<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="books.xsl"?>

<bookstore>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">XQuery Tips</title>
    <author>Tom Nolan</author>
    <year>2022</year>
    <price>39.99</price>
  </book>
  <book category="web">

```

```

<title lang="en">Learn XML</title>
<author>Tutorial Reference</author>
<year>2022</year>
<price>9.99</price>
</book>
</bookstore>

```

Step4: Finally you can now view the HTML

Close

The Book Store

Title	Author	Year	Price	Category
Harry Potter	J K. Rowling	2005	29.99	children
XQuery Tips	Tom Nolan	2022	39.99	web
Learn XML	Tutorial Reference	2022	9.99	web

Example 2:

```

<?xml version="1.0" encoding="UTF-8"?>

<student>
  <name>John Doe</name>
  <age>21</age>
  <address>
    <city>New York</city>
    <street>5th Avenue</street>
  </address>
  <email>johndoe@example.com</email>
</student>

```

Requirements:

1. H2 header showing Student details.
2. Create ul to show all the student details in element name: element value format.

Create student.xsl

```

<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/student">

```

```

<html>
  <body>
    <h2>Student Details</h2>
    <ul>
      <li>Name: <xsl:value-of select="name"/></li>

      <li>Age: <xsl:value-of select="age"/></li>

      <li>Address:
        <ul>
          <li>City: <xsl:value-of select="address/city"/></li>
          <li>Street: <xsl:value-of select="address/street"/></li>
        </ul>
      </li>
      <li>Email: <xsl:value-of select="email"/></li>
    </ul>
  </body>
</html>
</xsl:template>
</xsl:stylesheet>

```

Create student.xml with link of xsl.

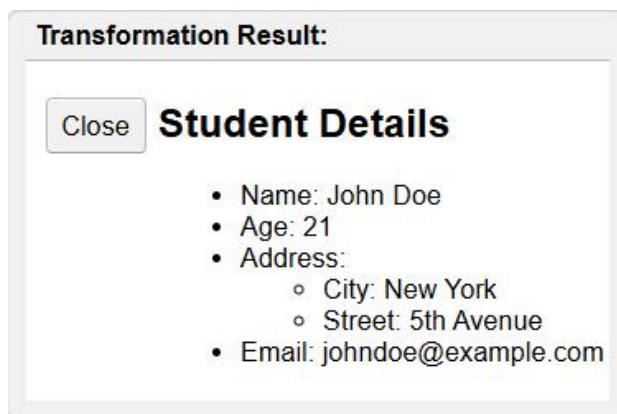
```

<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="student.xsl"?>

<student>
  <name>John Doe</name>
  <age>21</age>
  <address>
    <city>New York</city>
    <street>5th Avenue</street>
  </address>
  <email>johndoe@example.com</email>
</student>

```

Result



In the above example, root element is selected as `<xsl:template match="/student">` and all the other elements are referenced from this element. If we go from `/`, then we have to adjust the select.

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <html>
      <body>
        <h2>Student Details</h2>
        <ul>
          <li>Name: <xsl:value-of select="student/name"/></li>
          <li>Age: <xsl:value-of select="student/age"/></li>

          <li>Address:
            <ul>
              <li>City: <xsl:value-of select="student/address/city"/></li>
              <li>Street: <xsl:value-of
select="student/address/street"/></li>
            </ul>
          </li>
          <li>Email: <xsl:value-of select="student/email"/></li>
        </ul>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

<xsl:template>

`<xsl:template>` defines a way to reuse templates in order to generate the desired output for nodes of a particular type/context.

The `match` attribute is used to associate a template with an XML element. The `match` attribute can also be used to define a template for the entire XML document. The value of the `match` attribute is an XPath expression (i.e. `match="/"` defines the whole document).

<xsl:value-of>

The `<xsl:value-of>` element can be used to extract the value of an XML element and add it to the output stream of the transformation:

```
<xsl:value-of select="student/name"/>
<xsl:value-of select="title"/>
```

Note: The `select` attribute, in the example above, contains an XPath expression. An XPath expression works like navigating a file system; a forward slash (`/`) selects subdirectories.

<xsl:for-each>

The XSL `<xsl:for-each>` element can be used to select every XML element of a specified node-set:

```
<xsl:for-each select="bookstore/book">
```

We can also filter the result by giving some specific criteria using XPath.


```
<xsl:for-each select="bookstore/book[price<1000]">
<xsl:for-each select="bookstore/book@category=web">
```

<xsl:sort>

To sort the output, simply add an <xsl:sort> element inside the <xsl:for-each> element in the XSL file:

For example, in the above bookstore example, if we want to sort according to price, we adjust like following in xsl file.

```
<?xml version="1.0" encoding="UTF-8"?>

<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
  <html>
    <body>
      <h2>The Book Store</h2>
      <table border="1">
        <tr bgcolor="#6565d5">
          <th>Title</th>
          <th>Author</th>
          <th>Year</th>
          <th>Price</th>
          <th>Category</th>
        </tr>
        <xsl:for-each select="bookstore/book">
          <xsl:sort select="price"/>
          <tr>
            <td><xsl:value-of select="title"/></td>
            <td><xsl:value-of select="author"/></td>
            <td><xsl:value-of select="year"/></td>
            <td><xsl:value-of select="price"/></td>
            <td><xsl:value-of select="@id" /></td>
          </tr>
        </xsl:for-each>
      </table>
    </body>
  </html>
</xsl:template>

</xsl:stylesheet>
```

<xsl:if>

To add a conditional test, add the <xsl:if> element inside the <xsl:for-each> element in the XSL file:

Suppose we want to select all the elements if price is greater than 500.

```
<?xml version="1.0" encoding="UTF-8"?>

<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

```

<xsl:template match="/">
  <html>
    <body>
      <h2>The Book Store</h2>
      <table border="1">
        <tr bgcolor="#6565d5">
          <th>Title</th>
          <th>Author</th>
          <th>Year</th>
          <th>Price</th>
          <th>Category</th>
        </tr>
        <xsl:for-each select="bookstore/book">
          <xsl:if test = "price > 500">
            <tr>
              <td><xsl:value-of select="title"/></td>
              <td><xsl:value-of select="author"/></td>
              <td><xsl:value-of select="year"/></td>
              <td><xsl:value-of select="price"/></td>
              <td><xsl:value-of select="@id" /></td>
            </tr>
          </xsl:if>
        </xsl:for-each>
      </table>
    </body>
  </html>
</xsl:template>

</xsl:stylesheet>

```

XQuery

XQuery is a language for finding and extracting elements and attributes from XML documents. It is to XML what SQL is to database tables and is designed to query XML data. XQuery is a functional language that is used to retrieve information stored in XML format. XQuery can be used on XML documents, relational databases containing data in XML formats, or XML Databases.

- XQuery is the language for querying XML data.
- XQuery for XML is like SQL for databases.
- XQuery is built on XPath expressions.
- XQuery is supported by all major databases.
- XQuery is a W3C Recommendation since 2007.

Initial XML

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">XQuery Tips</title>
    <author>Tom Nolan</author>
    <year>2022</year>
    <price>39.99</price>
  </book>
  <book category="web">
    <title lang="en">Learn XML</title>
    <author>Tutorial Reference</author>
    <year>2022</year>
    <price>9.99</price>
  </book>
</bookstore>
```

XQuery Expressions

We want the title elements of those XML nodes where the price is greater than 10:

XQuery

```
for $x in doc("books.xml")/bookstore/book
where $x/price>10
return $x/title
```

Result

```
<title lang="en">Harry Potter</title>
<title lang="en">XQuery Tips</title>
```

Functions

XQuery uses functions to extract data from XML documents. The doc() function is used to open the "books.xml" file:

```
doc("books.xml")
```

XPath expressions

XQuery uses path expressions to navigate through elements in an XML document. The following path expression is used to select all the book elements in the "books.xml" file:

```
doc("books.xml")/bookstore/book
```

Predicates

XQuery uses predicates to limit the extracted data from XML documents. The following predicate is used to select all the book elements under the bookstore element that have a price element with a value that is less than 30:

```
doc("books.xml")/bookstore/book[price<30]
```

The XQuery above will extract the following:

```
<book category="children">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
<book category="web">
  <title lang="en">Learn XML</title>
  <author>Tutorial Reference</author>
  <year>2022</year>
  <price>9.99</price>
</book>
```

XQuery FLOWR

FLWOR is an acronym that stands for "**F**or, **L**et, **W**here, **O**rders by, **R**eturn".

The following list shows what they account for in a FLWOR expression:

- **For**: selects a sequence of nodes
- **Let**: binds a sequence to a variable
- **Where**: filters the nodes
- **Order by**: sorts the nodes
- **Return**: what to return (gets evaluated once for every node)

We will use a FLOWR expression to retrieve the authors of these books with a price lower than 20.

```
let $books := (doc("books.xml")/bookstore/book)
return <results>
{
  for $x in $books
  where $x/price<20
  order by $x/author
  return $x/author
}
</results>
```

- The **for** clause selects all book elements under the bookstore element into a variable called \$x.

- The **where** clause selects only book elements with a price element with a value lower than 20.
- The **order by** clause defines the sort-order. Will be sort by the author element.
- The **return** clause specifies what should be returned. Here it returns the title elements.

The result of the XQuery expression above is:

```
<author>Tutorial Reference</author>
```

Example 2:

```
for $x in doc("books.xml")/bookstore/book
return if ($x/@category="CHILDREN")
then <child>{data($x/title)}</child>
else <adult>{data($x/title)}</adult>
```

Output

```
<child>Harry Potter</child>
<adult>XQuery Tips</adult>
<adult>Learn XML</adult>
```

XQuery HTML Format

XQuery can also be used to transform an XML document into an HTML page.

We want to create an HTML table containing the titles of all books with their authors and prices:

```
let $books := (doc("books.xml")/books/book)
return <table><tr><th>Title</th><th>Author</th><th>Price</th></tr>
{
  for $x in $books
  order by $x/price
  return
  <tr><td>{data($x/title)}</td><td>{data($x/author)}</td><td>{data($x/price)}
  </td></tr>
}
</table>
```

The result of the XQuery expression above is:

```
<table>
  <tr>
    <th>Title</th>
    <th>Author</th>
    <th>Price</th>
  </tr>
```

```

<tr>
  <td>Harry Potter</td>
  <td>J K. Rowling</td>
  <td>29.99</td>
</tr>
<tr>
  <td>XQuery Tips</td>
  <td>Tom Nolan</td>
  <td>2022</td>
  <td>39.99</td>
</tr>
<tr>
  <td>Learn XML</td>
  <td>Tutorial Reference</td>
  <td>9.99</td>
</tr>
</table>

```

We want to list all the book titles in our bookstore in an HTML list.

```

<ul>
{
for $x in doc("books.xml")/bookstore/book/title
order by $x
return <li>{$x}</li>
}
</ul>

```

The result of the XQuery expression above is:

```

<ul>
<li><title lang="en">Harry Potter</title></li>
<li><title lang="en">XQuery Tips</title></li>
<li><title lang="en">Learn XML</title></li>
</ul>

```

[DTD – Attributes](#)

[advjhtml appB](#)

[XML DTD Attributes | Tutorial Reference](#)

[The Extensible Stylesheet Language Family \(XSL\)](#)

[XSLT Introduction](#)